

must take this into consideration. For the Z axis translation, we will use the values that we had already assigned while creating the game in the *setup()*.

Once drawn, we exit the matrix stack and update the values associated with the object by calling *move()* and *check_bounds()*. *move()* simply increases the Z axis value to make the object appear closer every time *draw()* is called. And the *check_bounds()* simply resets the Z axis value back to the initial point if it crosses the view of the player. It also changes the height of the obstacle to give the game a little more difficulty. On executing, you will see something similar to Figure 2.

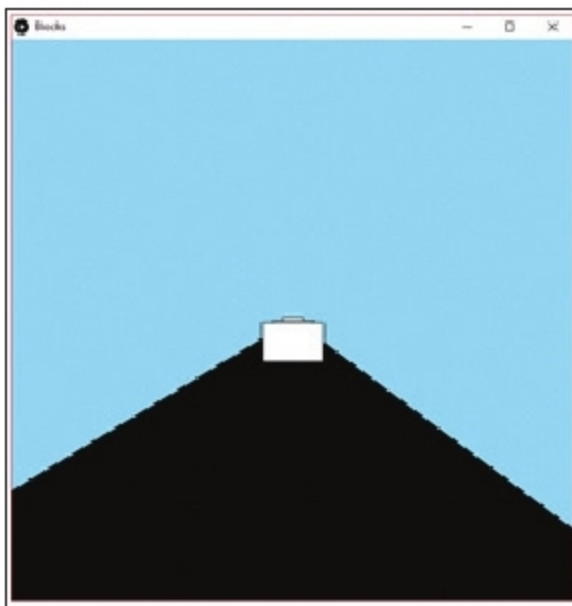


Figure 2: Obstacles

Putting the player in the game

Now that we have taken care of the obstacles and moving them along the path, we can shift our attention towards somehow recreating the first person view inside the game. In this tutorial, we will make use of *camera()*—by changing its parameters, we can emulate the game to look as if we are present in it.

Like always, let us start off by creating a player class, and shifting any variables related to the player there. In our case, we only have one variable, *fov*, and by taking it there, we can easily see what all are related to the player and can easily modify it.

```
class player:
    def __init__(self):
        self.j_flag = False
        self.grav_flag = True
```

```
self.eyeY = 1.5
self.ptY = 1.5
self.fov = PI/3
self.speed = 40
self.flag = 1
self.score = 0
self.highscore = 0
```

Now we can go on to defining the methods that we will use for the player. There are only three basic methods possible—one for jumping, one to bring the player back down and one to reset these parameters.

```
def jump(self):
    self.eyeY += 0.1
    self.ptY += 0.05

def grav(self):
    self.eyeY -= 0.1
    self.ptY -= 0.05

def reset(self):
    self.j_flag = False
    self.grav_flag = True
```

After defining an object, in this case *pl*, in *setup()*, we can jump right into *draw()* where the main part of the player-related logic will be written. But before adding in the extra jumping code, we must first edit *perspective()* as *fov* is defined in the player class. This can be easily done as shown below:

```
cameraZ = (height/2.0) / tan(pl.
fov/2.0)
perspective(pl.fov, float(width/2)/
float(height/2), 50, -cameraZ*20)
```

With that done, we must make sure that we can read the key presses on the player. For this, we will use the inbuilt *keyPressed()* to change the flag values that we have defined in the player class.

```
def keyPressed():
    global pl, gameOver, obs
    if key == CODED:
        if keyCode == UP and pl.flag == 1:
            pl.j_flag = True
            pl.grav_flag = False
            pl.flag += 1
```

Going back to *draw()*, we can now define the logic for jumping, coming back down and resetting the flags. It is also now that we will make use of *camera()* to emulate the player's eyes.

```
if pl.j_flag == True:
    pl.jump()

if pl.eyeY >= 4 and pl.ptY >= 3:
    pl.reset()

if pl.grav_flag == True:
    if pl.eyeY > 1.5 and pl.ptY >
1.5:
        pl.grav()
    else:
        pl.flag = 1

camera(width/2, height/pl.eyeY, 60,
width/2, height/pl.ptY, -100, 0, 1, 0)

lights()
```

Now, before going over the code, let us first see how *camera()* works. It takes in three sets of arguments:

- The eye position
- Scene centre
- Upwards axis

Note: We will only speak briefly about each one here in this tutorial; however, it is very important that you understand clearly how each of these affect the camera. Hence, it might be necessary for you to refer to the docs as mentioned earlier.

The eye position is exactly what the phrase implies. It tells Processing where you want your eye to be located, so changing this will affect how far up or down your view extends.

The scene centre tells Processing which point of the screen you are looking at. This changes with how near or far you are looking into the screen. For example, giving a high negative Z axis value makes it seem as if you are looking at the horizon.

The upwards axis changes which axis is pointing upwards and by default, in this game, it will be pointing upwards.